Antigravity QA Playbook — Orbin AI (E2E en Google Chrome)

Qué es esto: la secuencia exacta de prompts para pegar en **Google Antigravity** (Gemini 3) y que corra Orbin, la pruebe de punta a punta en Chrome —incluido el chat con IA—, corrija los errores que aparezcan y deje un registro de los prompts que usó.

Cómo usarlo: abrí Antigravity con la carpeta `Orbin` como workspace. Pegá los prompts **en orden** (P0 → P8). Cada prompt asume que el anterior terminó. Los prompts van en inglés a propósito (mejor rendimiento del modelo); las notas para vos van en español.

Anclado al estado real del repo (auditado 2026-07-21): - API backend: `http://localhost:3003` · App frontend: `http://localhost:5174` - Arranque: `npm run dev` en la raíz (usa `concurrently`) — levanta server + client juntos. - Chat IA: `POST /api/chat/design` → `aiOrchestrator.chatDesign` (cadena: **gemini-2.5-flash → vertex → ollama → parser regex**). - Motor: `server/src/engine/closetEngine.js` + `validator.js`. Suite: **90 tests**. - Galería: `POST /api/designs/save-viable` (regenera server-side antes de insertar) + `GET /api/designs/viable`. Tabla Supabase `viable_designs`. - Llaves en `server/.env` presentes: `GEMINI_API_KEY`, `GOOGLE_API_KEY`, `CLAUDE_API_KEY`, `SUPABASE_*` → chat IA y Galería funcionan live.

Pre-flight (hacelo vos, 2 min, antes de P0)

1. **Node ≥ 20** instalado (`node -v`). El repo lo exige (`engines.node >= 20`).
2. `server/.env` y `client/.env` existen y tienen las llaves. **No las pegues en el chat de Antigravity.**
3. Puertos `3003` y `5174` libres (cerrá cualquier `ARRANCAR.bat` previo).
4. Dependencias instaladas: si es primera vez, `npm run install:all` en la raíz.

P0 — Mission brief (rol + reglas duras)

```
You are a Senior QA & Full-Stack Engineer working inside the Orbin AI monorepo
(React+Vite client on :5174, Node/Express server on :3003, Supabase, Gemini/Claude).

MISSION: Boot the app, run a full end-to-end test in Google Chrome including the
AI design chat, fix any error you hit along the way, and keep a log of every
prompt you use.

NON-NEGOTIABLE RULES:
1. ADDITIVE ONLY. Do not delete or rewrite existing core logic in
   server/src/engine/*, validator.js, or aiOrchestrator.js. Fix by adding guards
   or narrow patches. Every change must keep `npm test` green (90 tests) and
   `npm run build` at 0 errors.
2. STRUCTURAL SAFETY. Furniture rules are hard limits: MDF plate max 2750x1840mm,
   drawer telescopic-slide clearance 13mm per side (26mm total), max unsupported
   MDF-18mm shelf span 900mm. No configuration that violates these may pass
   validation.
3. SECRETS. Never print, echo, or commit the contents of any .env file. Reference
   keys by name only.
4. VERIFY, DON'T ASSUME. After each fix, re-run the affected test before moving on.
5. LOG EVERYTHING. Maintain a file `PROMPTS_LOG.md` (see P8) — append every prompt
   I give you plus a one-line note of the action you took and the outcome.

Start by reading CLAUDE.md, README.md and server/src/routes/{chat,design}.js to
load context. Then confirm you understand the mission and list the test plan you
```

```
will follow. Do not start coding yet.
```

Nota (ES): obligás a Antigravity a leer contexto y devolver un plan *antes* de tocar nada. Si el plan omite el chat IA o la Galería, corregilo en el mismo turno.

P1 — Arranque y health-check

```
Boot the app and confirm both services are healthy.

Steps:
1. From the repo root, run `npm run dev` (it uses concurrently to start server
   on :3003 and client on :5174).
2. Wait until you see the server log "[Orbin] Server running on port 3003" and
   Vite's "Local: http://localhost:5174".
3. Health-check the API: GET http://localhost:3003/api/design/defaults — expect
   HTTP 200 with a JSON `defaults` object.
4. If either service crashes on boot, read the stack trace, diagnose the root
   cause, apply the smallest additive fix, and restart. Report what you changed.

Report: the exact URLs that are live, the API health response, and any fix applied.
```

Nota (ES): `/api/design/defaults` es público (sin auth) — ideal como health-check. Si el server no levanta, casi siempre es una llave faltante en `.env` o un puerto ocupado.

P2 — Smoke test en Chrome (landing → sesión)

```
Open Google Chrome (use your Browser tool) and smoke-test the shell.

1. Navigate to http://localhost:5174 and take a screenshot.
2. Verify the landing renders with no red console errors (open DevTools console
   and read it). List any console error verbatim.
3. Reach the main workspace (the parametric canvas / chat view). If auth blocks
   you, use the app's dev/guest path if one exists; otherwise create a test
   account. Note: the backend treats user id 'dev-local-user' specially.
4. Confirm the left panel (params) and the AI chat panel are both visible.

Report screenshots + any console error. Fix blocking errors (additive) and re-test.
```

Nota (ES): el backend reconoce `dev-local-user` para flujo local; Antigravity debería poder entrar sin Stripe. Si hay error de auth real, que lo documente pero no rompa el middleware.

P3 — Prueba del CHAT con IA (el núcleo del pedido)

```
Test the AI design chat end-to-end through the UI (not just the API).

Test conversation (type each message into the chat panel, wait for the AI reply,
screenshot each turn):

  Turn 1: "Quiero un armario de 1800mm de ancho, 2200 de alto y 580 de fondo,
           con 4 estantes y 3 cajones."
  Turn 2: "Agregale puertas y subí los cajones a 5."
  Turn 3: "Hacelo módulo aéreo de cocina, 800 de ancho y 350 de fondo."
```

```
For each turn verify:
- The request hits POST /api/chat/design (check the Network tab).
- The AI returns a coherent reply AND a `design` object (params + pieces), OR a
  clarifying question. It must NOT return a raw error string.
- The 3D viewer / cut list updates to match the described furniture.
- Turn 3 must produce an 'aereo' module with ZERO baseboard pieces (aéreos no
  llevan zócalo).

Then test the fallback chain resilience:
- Temporarily force the primary provider to fail (e.g. rename GEMINI_API_KEY in a
  LOCAL shell env only, never edit the .env file on disk) and confirm the chat
  still answers via the vertex→ollama→regex fallback without a 500 to the user.
  Restore the env immediately after.

Report: screenshots of all 3 turns, the Network payloads, and confirmation the
aéreo case dropped the baseboard. Fix any turn that errors (additive) and re-run.
```

Nota crítica (ES): el punto más valioso es el **Turno 3** (aéreo sin zócalo) y la **cadena de fallback**. Si Antigravity fuerza el fallo de la llave, que lo haga en una variable de entorno de shell temporal — **nunca editando `server/.env` en disco**. Reforzá esa regla si ves que va a tocar el archivo.

P4 — Motor paramétrico + Viewer 3D (casos borde)

```
Test the parametric engine through the UI params panel with 3 adversarial cases.
For each: enter the params, generate, and read the validation report.

CASE A (must PASS): base cabinet W800 H870 D580, moduleType 'base', 2 drawers.
  Expect: VALIDADO, 0 errors. Open the cut list and confirm drawer bottom =
  interior width minus 26mm (13mm slide clearance per side). For W800/18mm walls:
  interior 764 → drawer bottom 738mm.

CASE B (must PASS): airborne cabinet W800 H600 D320, moduleType 'aereo', 2 shelves.
  Expect: VALIDADO, 0 baseboard pieces, no false depth warnings.

CASE C (must be REJECTED): W3000 H2600. Expect: RECHAZADO with plate-size
  violations (pieces exceed 2750mm) and critical PIECE_EXCEEDS_PLATE adjustments.
  The UI must clearly surface the rejection, not silently pass.

Report the validation status + error count for each. If the UI shows a hardcoded
"all checks passed" instead of the real validator output, that is a BUG — fix it.
```

Nota (ES): estos son los mismos 3 casos que yo ya verifiqué a nivel motor (pasan). Acá el foco es que **la UI refleje fielmente** el veredicto del validador, no un mensaje hardcodeado.

P5 — Galería (persistencia) + Clipboard Ctrl+C / Ctrl+V

```
Test the Verified Gallery pipeline end-to-end.

1. Generate a VALIDADO design (use Case A from P4). Trigger "Guardar como Diseño
   Viable". Verify it POSTs to /api/designs/save-viable and returns 201 with a
   saved row. Confirm in the Network tab that the server REGENERATES the design
   (it must not trust client geometry).
2. Open the Galería overlay. Confirm the saved design appears with its category
   badge and W×H×D. Verify it was fetched via GET /api/designs/viable.
```

```
3. Click "Copiar" on the card. Then click on the canvas (not an input) and press
   Ctrl+V. Confirm the global paste listener parses the clipboard JSON and
   re-hydrates the workspace via generateProject (a new module appears, undoable
   with Ctrl+Z).
4. Negative test: try to save Case C (W3000). It MUST be rejected with code
   NOT_VIABLE and must NOT appear in the gallery.

If Supabase is unreachable, save-viable returns 503 STORAGE_UNAVAILABLE by design
(no in-memory fallback). If you see 503, verify SUPABASE_URL/SERVICE_KEY are set
and report — do not add a fake fallback.

Report screenshots of: the saved card, the successful Ctrl+V rehydration, and the
rejected Case C.
```

Nota (ES): el guard anti-forja (regenerar server-side) es correcto y no se toca. El 503 es comportamiento *intencional*, no un bug — si aparece, es config de Supabase, no código.

P6 — Protocolo de corrección de errores (loop)

```
For EVERY error encountered in P1–P5, follow this loop and log it:

1. REPRODUCE: capture the exact error (console, network, or server stack trace).
2. ISOLATE: identify the single root cause. State it in one sentence.
3. FIX (additive): apply the smallest change that resolves it without altering
   engine/validator core behavior. Prefer guards, null-checks, and mapping fixes.
4. VERIFY: re-run the specific failing step AND `cd server && npm test`. Both
   must pass.
5. LOG: append to PROMPTS_LOG.md — error, root cause, fix (file + lines), result.

Do NOT batch risky changes. One fix, one verification. If a fix would require
rewriting core logic, STOP and report it as a recommendation instead of doing it.
```

P7 — Compuerta de regresión (antes de cerrar)

```
Final regression gate. Run and report all three:

1. `cd server && npm test`  → must be 90/90 passing, 0 fail.
2. `npm run build` (root, builds the client) → must exit 0 with no errors.
   Note: the client bundle is heavy (three.js + jspdf); allow it up to ~3 minutes.
3. Re-open http://localhost:5174 in Chrome and confirm the app still boots clean
   after all fixes (no new console errors).

If any gate fails, fix (additive) and re-run all three. Paste the final test
summary and the build's last 15 lines of output.
```

P8 — Documento de prompts (el entregable que pediste)

```
Create/finalize PROMPTS_LOG.md at the repo root. Structure:

# Orbin QA — Prompts Log (Antigravity run)
- Date, Antigravity/Gemini version, machine.
```

```
## Prompt sequence
For each prompt P0–P8 I gave you:
  ### P{n} — {title}
  - Prompt (verbatim, first 3 lines + "...").
  - Action taken.
  - Result (pass/fail).
  - Files changed (path + line ranges), if any.

## Errors found & fixed
Table: Error | Root cause | Fix (file:lines) | Verified?

## Final regression
- npm test result (X/90).
- npm run build result (exit code).
- Chrome smoke re-test result.

## Screenshots
Reference the screenshots you captured (P2, P3, P4, P5).

## Open recommendations
Anything you chose NOT to fix because it needed core rewrites.

Save the file and show me its full contents.
```

Nota (ES): este es el documento que Antigravity te deja. Si querés, guardalo también como PDF al final para archivarlo.

Guardrails resumidos (pegá esto si Antigravity se desvía)

```
Reminder of hard limits:
- Additive changes only. Keep 90/90 tests green and build at 0 errors.
- Never edit or print any .env file. Force provider failures via shell env only.
- Plate 2750x1840mm, slide clearance 13mm/side, shelf span ≤900mm are inviolable.
- 503 STORAGE_UNAVAILABLE and NOT_VIABLE rejections are correct behavior, not bugs.
- One fix → one verification. No batched risky edits.
```

Riesgo latente (lo que puede salir mal)

- **Antigravity intenta “arreglar” el validador para que Caso C pase.** Es el error clásico de un agente que confunde “rechazo” con “bug”. Por eso P4/P5 declaran explícitamente que el rechazo es el resultado correcto. Vigilá esto.
- **Toca `server/.env` para probar el fallback.** Prohibido en P3/guardrails. Si lo hace, revertí y reforzá.
- **El build de 3 min lo interpreta como “colgado”** y lo mata. P7 le avisa del tiempo esperado.
- **Chat IA sin red / cuota Gemini agotada** → cae a Ollama (necesita Ollama local corriendo) o al parser regex. Si no tenés Ollama, el fallback final es regex: la respuesta será más pobre pero no debe dar 500.